

Book Shelf Installation and Release Guide

Developed and maintained by **Saima Usman**.

This guide covers local Docker Compose, managed Northflank/Aiven deployment, and the optional AWS EC2 infrastructure. No fork is required. All cloud provisioning and release operations are explicit operator actions.

1. Select a Deployment Model

Model	Components	Intended use
Local Compose	Nginx, Next.js, Express, MySQL	Development and isolated tests
Northflank + Aiven	Two application services and managed MySQL	Managed public deployment
EC2 Compose	Four-service stack on Ubuntu x86_64	Self-managed alternative

Public deployments require HTTPS, protected credentials, backups, and operational monitoring. Free tiers have resource/availability restrictions and do not imply an uptime guarantee. Review the [production checklist](#).

2. Clone and Install Prerequisites

```
git clone https://github.com/Saima-Devops/Book-Review-App.git
cd Book-Review-App
```

Use Git and Docker with Compose v2 for local containers. Node.js 22 and npm are required for direct development/tests; Python 3 supports configuration tests. OpenSSL generates secrets. Terraform, Ansible, AWS CLI v2, and OpenSSH are needed only for optional EC2 provisioning.

3. Local Docker Compose

```
cp .env.example .env
chmod 600 .env
```

Set `PUBLIC_URL=http://localhost`. Replace `MYSQL_ROOT_PASSWORD`, `MYSQL_PASSWORD`, and `JWT_SECRET` with three different random values, each generated privately with `openssl rand -hex 32`. Retain database/user identifiers and the existing `DB_VOLUME_NAME` when upgrading. Leave optional email/admin settings empty until configured.

```
docker compose config --quiet
docker compose up -d --build --wait --wait-timeout 600
docker compose ps
PUBLIC_URL=http://localhost bash scripts/verify.sh
```

Open `http://localhost`. Only Nginx publishes port 80. An initial empty database receives sample books; existing records are retained. MySQL data remains in `reading-room_db_data` by default. Never delete volumes during routine releases.

Stop without removing data:

```
docker compose down
```

Port 80 must be available. If occupied, use a reviewed Compose override for a different host port and set `PUBLIC_URL` to the matching origin. Do not expose MySQL publicly. Public HTTP must not carry real credentials.

4. Direct Local Development

Start a local MySQL 8.4 database and application account, with privileges scoped to the application's database. Create `backend/.env`, excluded from Git:

```
PORT=3001
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=book_review_db
DB_USER=bookreview_user
DB_PASS=LOCAL-DATABASE-PASSWORD
JWT_SECRET=LOCAL-RANDOM-SIGNING-SECRET
ALLOWED_ORIGINS=http://localhost:3000
DB_SSL=false
ADMIN_USER_IDS=
```

Create `frontend/.env.local` containing `NEXT_PUBLIC_API_URL=http://localhost:3001`. Run the backend and frontend in separate terminals from the repository root:

```
npm --prefix backend ci
npm --prefix backend start

npm --prefix frontend ci
npm --prefix frontend run dev
```

Open `http://localhost:3000`. Local development must not use production database credentials or a live database for integration tests.

5. Managed Database Preparation

Create or retain an Aiven MySQL service and the application database `book_shelf`. Download the service/project CA certificate. The Overview connection example may retain `defaultdb`; clients select `book_shelf` independently.

For existing application data, take a protected logical backup and import into an empty target before starting the backend. Preserve user IDs with books and reviews so uploader ownership remains valid. Verify counts and representative records; do not publish exports or passwords.

See [backup and restore](#). The Compose scripts operate on local containers, not Aiven. Provider backups supplement, rather than replace, tested independent recovery copies.

Prefer a dedicated application database account over the service administrator. Allow privileges needed for application queries and additive startup migrations. Use verified TLS and an allowlist covering the hosted backend's egress.

6. Publish a Release

Push the reviewed release to main. Wait for all GitHub Actions jobs and the Docker Hub publishing step to complete. Record the commit, run/attempt, and frontend/backend image digests.

The registry repository is `saim2026/book-shelf`. Select matching `frontend-build-RUN-ATTEMPT` and `backend-build-RUN-ATTEMPT` tags from the workflow summary. Latest aliases are not immutable release

references. No cloud rollout is triggered by the publishing workflow.

See [CI and delivery](#).

7. Deploy the Northflank Backend

Create or update a deployment service from the tested backend image. Retain the startup command. Configure one appropriately sized instance and public HTTP container port 3001. Northflank provides HTTPS at the public entry point.

Set backend runtime values:

```
NODE_ENV=production
PORT=3001
DB_HOST=AIVEN-SERVICE-HOSTNAME
DB_PORT=AIVEN-SERVICE-PORT
DB_NAME=book_shelf
DB_USER=APPLICATION-DATABASE-ACCOUNT
DB_PASS=SECRET-DATABASE-PASSWORD
DB_SSL=true
DB_SSL_CA_FILE=/app/certs/ca.pem
JWT_SECRET=STABLE-RANDOM-SIGNING-SECRET
ADMIN_USER_IDS=
```

Store passwords/signing secrets through protected hosting configuration. Mount `ca.pem` as a runtime file at `/app/certs/ca.pem`, readable by the runtime user. Leave `DB_SSL_CA` unset. Inline PEM is an alternative, not an additional CA setting. Never disable certificate/hostname verification.

Set platform health checks to HTTP port 3001 at `/api/books`. A root `/` check tests the process but does not continually verify MySQL. Allow schema initialization time. Verify database connection and server startup in runtime logs.

Aiven allowlists must permit Northflank egress. Restricting access to a laptop address prevents the backend from connecting. Open IPv4/IPv6 ranges are broad exposure, not finished network hardening. Check stable-egress availability/cost.

8. Deploy the Northflank Frontend

Use the matching frontend image, unchanged startup command, and public HTTP container port 3000. Configure runtime values:

```
NODE_ENV=production
PORT=3000
BACKEND_API_ORIGIN=https://BACKEND-PUBLIC-HOSTNAME
```

The backend origin must omit `/api`, `:3001`, credentials, query, and fragment. The published frontend already uses same-origin `/api` requests; its server route forwards them to the configured backend. No separate Nginx service or Northflank database add-on is needed.

Set frontend HTTP health checks to `/` on port 3000. Do not place database credentials, CA files, SMTP credentials, or JWT secrets in the frontend.

9. Configure Optional Features

Book covers: no additional environment variables or storage service are required. The backend adds nullable cover columns to Books on startup. Back up before the first deployment, and ensure the database account can add columns. Signed-in readers can preview JPG, PNG, or WebP images up to 5 MB when adding a book. Covers are resized and included in database backups. See [book covers](#) for storage and validation limits.

Administrator access: identify account IDs in the deployment database and set backend ADMIN_USER_IDS to the comma-separated IDs. Empty disables moderation. The Moderation link appears for authorized accounts; its route is /admin/reports. See [moderation](#).

Password recovery: configure backend SMTP settings and PUBLIC_URL with the frontend HTTPS origin. Delivery is unavailable until configured and tested. See [password recovery](#).

Custom domain: verify domain ownership in Northflank, add the supplied DNS records, and link the domain to frontend port 3000. Wait for the managed certificate. Update backend PUBLIC_URL and test reset links after domain changes. A URL shortener redirects but does not replace the address in the browser. [Northflank domains](#).

10. Validate the Release

```
export PUBLIC_URL=https://FRONTEND-PUBLIC-HOSTNAME
curl --fail --silent --show-error "$PUBLIC_URL/" >/dev/null
curl --fail --silent --show-error "$PUBLIC_URL/api/books"
curl --fail --silent --show-error "$PUBLIC_URL/login" >/dev/null
```

Check expected records, existing login, book details, reviews, favourites, and ownership restrictions. Moderation removal/recovery tests belong on disposable content in an isolated environment, not real user books. Review logs for errors without displaying secrets. Record deployed tags/digests and verification results.

Keep the previous compatible image references available for rollback. Do not automatically restore a database during image rollback. Deployment is not highly available by default; restarts may interrupt service.

11. Optional AWS EC2 Deployment

This path is separate from Northflank. Configure AWS credentials locally, then collect infrastructure values in Bash:

```
bash
source scripts/runtime-inputs.sh
bash scripts/deploy.sh plan
```

The helper collects region at runtime, x86_64 instance type, numeric root volume GiB, SSH IPv4 /32, private/public key paths, pushed commit, repository, deployment directory, secret source, and operator label FULL_NAME. Use existing keys without overwriting them. Verify cost/credit eligibility independently of instance names.

Review the saved Terraform plan for replacement/deletion before applying. Back up any existing EC2 database before disk/instance replacement.

```
bash scripts/deploy.sh apply
```

Apply provisions infrastructure only. Verify the SSH host fingerprint through a trusted AWS channel before accepting it; strict host-key checking remains enabled.

```
bash scripts/deploy.sh prepare
```

Prepare installs host tools and checks out the pushed commit. It does not start application containers. Reconnect after Docker group changes. Configure a protected environment on the host with python3 scripts/configure-env.py, retaining database passwords/volume names on upgrades. Then run from the configured workstation:

```
bash scripts/deploy.sh configure
```

Configure validates and starts the stack; it does not provision infrastructure. The base entry point is HTTP port 80. Optional HTTPS requires the overlay and renewal operations in [the IP TLS guide](#). Public application ports 3000/3001 and database port 3306 remain closed.

AWS resources can incur charges. The helper has no destroy subcommand. Infrastructure decommissioning requires a separately reviewed Terraform destroy plan from the workstation, an off-host data backup, and protected state retention.

12. Optional AWS Secrets Manager

1. In the selected AWS region, create an Other type of secret.
2. Store MYSQL_DATABASE, MYSQL_USER, MYSQL_ROOT_PASSWORD, MYSQL_PASSWORD, and JWT_SECRET.
3. Use distinct random secrets; optional SMTP fields may be included.
4. Select the AWS-managed Secrets Manager key or configure a scoped custom KMS policy.
5. Export SECRET_SOURCE=aws and SECRET_ARN before the infrastructure plan.
6. Confirm the generated EC2 role can read only the intended secret.
7. On EC2, set AWS_REGION and SECRET_ARN and run the environment helper.

The EC2 role supplies AWS access; application secret values are not Terraform variables. Retrieval materializes a mode-0600 .env. Docker administrators can inspect runtime environments. Generic secret updates do not rotate MySQL accounts; rotation requires coordinated database and configuration changes. Secret storage and infrastructure may incur charges.

13. Maintenance and Recovery

Keep independent encrypted backups, test restoration, monitor resource availability, and rotate credentials. Retain application data during releases. The Reports table is additive; moderation snapshots have privacy/retention implications. Favourites belong to each browser origin and do not migrate through SQL.

See [operations](#), [logging](#), and [production readiness](#).

14. Maintain the PDF Edition

The Markdown guide is the canonical source. Regenerate installation_guide.pdf with python3 scripts/export-installation-guide.py in a Python environment with ReportLab installed. Review the rendered pages and commit both editions together.